

Test-case-v1.0

Test-case-v1.0

HELLO BABY



Ομάδα

Όνομα	Επώνυμο	ΑΜ	email
Κρίστελ	Ντουλάϊ	1064498	up1064498@ac.upatras.gr
Ανδρέας	Γιαννικόπουλος	1067393	up1067393@ac.upatras.gr
Κλεοπάτρα	Πελέκη	1072557	up1072557@ac.upatras.gr
Ιωάννα	Νικολακοπούλου	1072609	up1072609@ac.upatras.gr
Βασιλική	Παγώνη	1072497	up1072497@ac.upatras.gr

TEST CASE :

Parent :

1. [Προγραμματισμός ραντεβού](#)
2. [Πληρωμές λογαριασμών](#)
3. [Συμμετοχή σε σεμινάρια](#)

Manager:

4. [Διαχείριση δωματίων](#)
5. [Διαχείριση σεμιναρίων](#)
6. [Εκδοση εξιτηρίου](#)

Doctor:

7. [Διαχείριση αιτημάτων για ραντεβου](#)
8. [Ενημέρωση ιατρικού φακέλου](#)
9. [Συνταγογράφηση](#)
10. [Διεξαγωγή εξετάσεων](#)

Test Case 1: Προγραμματισμός Ραντεβού (αρχεία parentAppointment_gui, και parentAppointment)

Η εφαρμογή υλοποιεί σύστημα κράτησης ραντεβού βασισμένο σε MVC με Tkinter. Ο `ParentAppointmentController` διαχειρίζεται τη ροή δεδομένων μεταξύ GUI και μοντέλων (`Doctor`, `Appointment`, `DoctorCalendar`). Η επιλογή κατηγορίας και γιατρού γίνεται με δυναμικά dropdowns, ενώ η διαθεσιμότητα slots ελέγχεται κεντρικά. Αποτροπές λαθών υλοποιούνται μέσω ελέγχων για κενές λίστες, προεπιλογών στις φόρμες (`StringVar`), και μηνυμάτων επιβεβαίωσης (`messagebox`). Η ειδοποίηση γιατρού γίνεται με την κλήση `NotificationService.notify()`, ενώ το σύστημα διασφαλίζει την ορθή αφαίρεση slots από το ημερολόγιο.

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη απο το να το κάνει	Μηχανισμός Αποτροπής στον κώδικα
1	Επιλογή κατηγορίας γιατρού χωρίς διαθέσιμους γιατρούς	Ενημερώνουμε ότι δεν υπάρχουν διαθέσιμοι γιατροί και δεν επιτρέπουμε συνέχεια	Εμφάνιση <code>AppointmentWaitListScreen</code> αντί <code>ChooseDoctorScreen</code> όταν η λίστα γιατρών είναι κενή
2	Προσθήκη στη λίστα αναμονής χωρίς επιλογή κατηγορίας	Η επιλογή κατηγορίας είναι απαραίτητη πριν προσθέσουμε στη λίστα	<code>requestAddToWaitList</code> δέχεται κατηγορία, δεν εκτελείται αν δεν υπάρχει

3	Επιλογή γιατρού χωρίς να έχει φορτωθεί η λίστα γιατρών	Δεν εμφανίζεται επιλογή αν η λίστα είναι κενή	<code>ChooseDoctorScreen</code> δεν δημιουργείται αν η λίστα είναι κενή (λογική ελέγχου πριν εμφάνιση)
4	Επιλογή slot που δεν είναι διαθέσιμο	Η λίστα slot προέρχεται μόνο από τα διαθέσιμα slot, ο χρήστης δεν μπορεί να επιλέξει κάτι άλλο	<code>DoctorCalendar.getAvailableSlots</code> επιστρέφει μόνο διαθέσιμα slots που εμφανίζονται στο dropdown
5	Δημιουργία ραντεβού χωρίς επιλογή slot	Δεν επιτρέπουμε κλικ στο "Αποθήκευση" αν δεν υπάρχει επιλογή slot	<code>selected_slot</code> έχει default τιμή (πρώτο διαθέσιμο slot), διασφαλίζοντας επιλογή
6	Ακύρωση κατά την επιλογή slot	Επιβεβαίωση ακύρωσης και επιστροφή στην αρχική οθόνη	<code>BookAppointmentScreen.redirect</code> καταστρέφει το παράθυρο και ξεκινά την αρχική οθόνη

Test case 2: Πληρωμές Λογαριασμών (αρχεία payment_gui, paymentController)

Η εφαρμογή υλοποιεί λειτουργικότητα πληρωμών με χρήση του προτύπου MVC μέσω της βιβλιοθήκης Tkinter. Ο `PaymentController` διαχειρίζεται την επιλογή και επιβεβαίωση λογαριασμών, τη μετάβαση μεταξύ οθονών και τη διασύνδεση με τα μοντέλα (`Payment`). Η διαδικασία πληρωμής περιλαμβάνει βήματα επιλογής λογαριασμών (`checkboxes`), προβολής του συνολικού ποσού και επιλογής τρόπου πληρωμής (`radio buttons`). Επιβεβαίωση πληρωμής γίνεται με `messagebox.askokcancel`, ενώ σε περίπτωση αποτυχίας εμφανίζεται κατάλληλο παράθυρο επαναπροσπάθειας. Αποτροπή λαθών εξασφαλίζεται μέσω ελέγχων για άδειες επιλογές, προειδοποιητικών μηνυμάτων (`messagebox`), καθώς και με χρήση default τιμών και διαχωρισμένων οθονών για κάθε στάδιο της διαδικασίας.

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη	Μηχανισμός Αποτροπής στον Κώδικα
1	Προβολή λογαριασμών χωρίς εκκρεμείς πληρωμές	Απλώς εμφανίζεται κενή λίστα, δεν επιτρέπεται επιλογή	<code>fetchPendingBills</code> επιστρέφει κενή λίστα — δεν δημιουργούνται <code>Checkbutton</code>

2	Συνέχεια χωρίς επιλογή λογαριασμού	Ο controller δεν προχωρά αν δεν υπάρχουν επιλεγμένα	<code>sendSelectedBills → selected_bills = []</code> , δεν εμφανίζεται επόμενο βήμα
3	Πληρωμή χωρίς επιλογή τρόπου πληρωμής	Εμφανίζεται προειδοποίηση και διακόπτεται η ροή	<code>save</code> στην <code>ViewAmountScreen</code> : <code>if not method → messagebox.showwarning</code>
4	Επιβεβαίωση πληρωμής χωρίς επιβεβαίωση από χρήστη	Ο χρήστης πρέπει να απαντήσει σε dialog	<code>messagebox.askokcancel(...)</code> στην <code>Confirm2Screen</code> πριν κληθεί <code>processPayment</code>
5 *	Επαναπροσπάθεια πληρωμής μετά από αποτυχία	Εμφανίζεται νέο παράθυρο με "Ναι" και "Όχι"	<code>RetryMessScreen</code> εμφανίζεται όταν <code>fail_pass == False</code> — κουμπιά με επαναφορά
6	Επιλογή πολλαπλών λογαριασμών και άθροισμα ποσού	Το σύστημα υπολογίζει άθροισμα πριν πληρωμή	<code>addAmount()</code> καλείται από <code>checkNumberOfSelectedBills</code> όταν επιλεγούν >1
7	Εμφάνιση εσφαλμένου ποσού στην πληρωμή	Χρησιμοποιούνται μόνο οι επιλεγμένοι λογαριασμοί	<code>sendSelectedBills()</code> → κρατά μόνο τα επιλεγμένα <code>bill_ids</code>

* Σχετικά με αυτή την περίπτωση να αναφέρουμε πως στα use cases η αποτυχία πληρωμής μπορεί να οφείλεται σε λόγους εξωτερικούς από το σύστημα μας, όπως το να μην δουλεύει το ebanking του χρήστη ή να μην δέχεται την χρέωση η τράπεζα. Ακριβώς επειδή η επικοινωνία γίνεται εξωτερικά για να προσομοιώσουμε την αποτυχία/επιτυχία της πληρωμής χρησιμοποιούμε την μεταβλητή `fail_pass` μέσα στην μέθοδο του `paymentController` αρχείου `processPayment()`, την οποία μεταβλητή την αλλάζουμε χειροκίνητα. Όταν είναι `True` ο έλεγχος `if-else` θεωρεί ότι η πληρωμή ήταν επιτυχημένη ενώ όταν είναι `False` ο έλεγχος θεωρεί πως ήταν αποτυχημένη. Ανάλογα σε ποιά από τις δύο περιπτώσεις βρισκόμαστε ακολουθούνται και οι ανάλογες ενέργειες που περιγράφονται.

Test case 3: Συμμετοχή σε Σεμινάρια (αρχεία `parentSeminar_gui`, `parentSeminar`)

Η εφαρμογή επιτρέπει στους γονείς να δουν διαθέσιμα σεμινάρια και να δηλώσουν συμμετοχή. Η λογική υλοποιείται με τη χρήση MVC σε Tkinter. Ο `BookSeminarController`` χειρίζεται τις ροές μεταξύ GUI και μοντέλων (`Seminar``, `SeminarParticipants``, `NotificationWaitList``) και εξασφαλίζει την ομαλή μετάβαση οθονών. Αν δεν υπάρχουν διαθέσιμα σεμινάρια, ο χρήστης ενημερώνεται μέσω `FMessage3Screen``, ενώ υποστηρίζεται και προσθήκη στη λίστα αναμονής. Σε κάθε βήμα, εφαρμόζονται έλεγχοι διαθεσιμότητας, επιβεβαιώσεις συμμετοχής και προστασία από ελλιπή ή μη έγκυρη είσοδο. Οι ειδοποιήσεις διαχειρίζονται από το `NotificationService``.

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη από το να το κάνει	Μηχανισμός Αποτροπής στον Κώδικα
1	Προσπάθεια εμφάνισης λίστας σεμιναρίων ενώ δεν υπάρχουν διαθέσιμα	Εμφανίζεται κατάλληλο μήνυμα και εναλλακτική για ειδοποίηση	<code>`if not seminars:`</code> → <code>`FMessage3Screen`</code> με μήνυμα "Δεν υπάρχουν διαθέσιμα σεμινάρια"
2	Δήλωση συμμετοχής χωρίς να επιλεγεί κάποιο σεμινάριο από τη λίστα	Εμφανίζεται προειδοποίηση ότι πρέπει να γίνει επιλογή	<code>`selectSeminar()`</code> ελέγχει με <code>try-except</code> για <code>IndexError</code> και εμφανίζει <code>`messagebox.showwarning`</code>
3	Προσπάθεια συμμετοχής σε πλήρες σεμινάριο	Εμφανίζεται μήνυμα και αποτρέπεται η διαδικασία δήλωσης	<code>`Seminar.getAvailability()`</code> → εάν <code>False</code> , εμφάνιση <code>`messagebox.showinfo("Πλήρες")`</code>
4	Επανάληψη συμμετοχής σε ήδη πλήρες σεμινάριο (ενώ προηγήθηκε έγκυρη επιλογή)	Ενημερώνεται ότι είναι πλήρες και αποτρέπεται νέα συμμετοχή	<code>`actOnParticipation()`</code> → αν <code>`updateParticipantList`</code> επιστρέψει <code>False</code> , δείχνει μήνυμα
5	Προσπάθεια για ειδοποίηση χωρίς προσθήκη στη λίστα	Προσθήκη γίνεται μόνο εφόσον ο χρήστης πατήσει το αντίστοιχο κουμπί	<code>`chooseNotification()`</code> → κλήση <code>`addToNotificationWaitList()`</code> μόνο εφόσον επιλεγθεί
6	Επιβεβαίωση συμμετοχής χωρίς ενδιάμεση οθόνη	Δεν είναι εφικτό καθώς η οθόνη επιβεβαίωσης εμφανίζεται μόνο αν υπάρχει διαθεσιμότητα	<code>`checkAvailability()`</code> οδηγεί σε <code>`Confirm3Screen`</code> μόνο αν υπάρχει διαθεσιμότητα
7	Ακύρωση δήλωσης στην οθόνη επιβεβαίωσης	Ο χρήστης επιστρέφει στην αρχική οθόνη χωρίς να καταχωρηθεί τίποτα	<code>`rejectParticipation()`</code> → καλεί <code>`ViewSeminarScreen.redirect()`</code>
8	Αποτυχία συμμετοχής λόγω μη ενημέρωσης του διαθέσιμου αριθμού	Μειώνεται διαθέσιμος αριθμός μόνο σε επιτυχημένη συμμετοχή	<code>`updateAvailability()`</code> καλείται **μόνο** όταν <code>`updateParticipantList()`</code> επιστρέψει <code>True</code>

Test case 4: Διαχείριση Δωματίων (αρχεία managerAppointment_gui, managerAppointment)

Στην εφαρμογή διαχείρισης ραντεβού έχουν υλοποιηθεί συγκεκριμένες τεχνικές ώστε να αποτρέπονται σφάλματα από την πλευρά του χρήστη και να διασφαλίζεται η ομαλή ροή της διαδικασίας. Χρησιμοποιούνται μηχανισμοί ελέγχου όπως οι δομές `try-except`, οι οποίες αποτρέπουν την εμφάνιση σφαλμάτων σε περιπτώσεις που ο χρήστης δεν επιλέγει αντικείμενα από λίστες, ενώ ταυτόχρονα εμφανίζονται προειδοποιητικά μηνύματα (`messagebox.showwarning`) που καθοδηγούν τον χρήστη στο τι πρέπει να κάνει. Επιπλέον, όταν δεν υπάρχουν διαθέσιμοι πόροι (όπως δωμάτια ή νοσηλευτές), εμφανίζονται εναλλακτικές οθόνες με σαφείς οδηγίες και επιλογές (όπως λίστα αναμονής ή ειδοποίηση γονέα), αποτρέποντας τον χρήστη από το να προχωρήσει χωρίς τα απαιτούμενα δεδομένα. Τέλος, η επιβεβαίωση ή απόρριψη των ενεργειών γίνεται με σαφείς επιλογές, προσφέροντας στον χρήστη τον απαραίτητο έλεγχο και αποτρέποντας ακούσια αποθήκευση ή ενέργειες χωρίς συναίνεση.

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη απο το να το κάνει	Μηχανισμός Αποτροπής στον κώδικα
1	Ο χρήστης προσπαθεί να συνεχίσει χωρίς να επιλέξει ραντεβού	Εμφάνιση μηνύματος: "Παρακαλώ επιλέξτε ένα ραντεβού."	<code>try-except IndexError + messagebox.showwarning()</code> στο <code>chooseAppointment()</code>
2	Ο χρήστης δεν επιλέγει δωμάτιο και πατάει "Επόμενο"	Εμφάνιση μηνύματος: "Παρακαλώ επιλέξτε δωμάτιο."	<code>try-except IndexError + messagebox.showwarning()</code> στο <code>chooseRoom()</code>
3	Δεν υπάρχουν διαθέσιμα δωμάτια	Εμφάνιση παραθύρου με επιλογή προσθήκης στη λίστα αναμονής	Εμφάνιση <code>AssignmentWaitingListScreen</code> όταν <code>rooms</code> είναι κενό
4	Χρήστης δεν επιλέγει κανέναν νοσηλευτή	Εμφάνιση μηνύματος: "Παρακαλώ επιλέξτε τουλάχιστον έναν νοσηλευτή."	<code>if not indices: + messagebox.showwarning()</code> στο <code>chooseNurses()</code>
5	Δεν υπάρχουν διαθέσιμοι νοσηλευτές	Εμφάνιση παραθύρου ειδοποίησης και αποστολή μηνύματος σε γονέα	<code>NurseNotificationScreen + notifyParentNoNurses()</code>

6	Ο χρήστης επιλέγει "Απόρριψη" στην επιβεβαίωση ανάθεσης	Δωμάτιο επιστρέφει ως διαθέσιμο	<code>callControllerForReject()</code> καλεί <code>updateRoomAsAvailable()</code>
7	Ο χρήστης πατάει "Αποθήκευση" στην επιβεβαίωση ανάθεσης	Αποθήκευση ανάθεσης, ενημέρωση νοσηλευτών, αποστολή ειδοποίησης	<code>callControllerForSave()</code> καλεί πολλαπλές υπηρεσίες
8	Ο χρήστης πατάει "Ακυρο" στη λίστα αναμονής	Κλείσιμο παραθύρου	Κλήση <code>self.root.destroy()</code> στο κουμπί ακύρωσης

Test case 5: Διαχείριση σεμιναρίων (αρχεία `seminarsMan`, `seminarsManager_gui`)

Η εφαρμογή υλοποιεί σύστημα διαχείρισης σεμιναρίων με χρήση Tkinter και αρχιτεκτονική MVC. Ο `ManagerSeminarController` ελέγχει την πλήρη ροή δημιουργίας, προβολής, αποθήκευσης και διαγραφής σεμιναρίων. Οι διαθέσιμες αίθουσες και ώρες φορτώνονται δυναμικά μέσω των κλάσεων `SeminarRoom` και `SeminarCalendar`. Κατά την αποθήκευση (`saveSeminar()`), το σεμινάριο καταχωρείται στο ημερολόγιο και αφαιρείται η αντίστοιχη ώρα από τη διαθεσιμότητα. Αν δεν υπάρχουν διαθέσιμες αίθουσες, εμφανίζεται κατάλληλο μήνυμα με `Message5Screen`. Σε διαγραφή (`callControllerForDelete()`), το σύστημα ειδοποιεί αυτόματα τους συμμετέχοντες μέσω `NotificationService` και επαναδιαθέτει την ώρα. Περιπτώσεις πλήρους πληρότητας οδηγούν σε προσθήκη του σεμιναρίου σε λίστα αναμονής (`SeminarRoomWaitingList`).

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη απο το να το κάνει	Μηχανισμός Αποτροπής στον κώδικα
1	Ο διαχειριστής προσπαθεί να επιλέξει αίθουσα ενώ δεν υπάρχουν	Εμφανίζεται κατάλληλο μήνυμα και δεν επιτρέπεται συνέχεια	Μέθοδος <code>fetchSeminarRoom()</code> : αν <code>rooms</code> είναι άδειο, καλεί <code>Message5Screen</code>
2	Ο χρήστης προσπαθεί να διαλέξει ημερομηνία που δεν υπάρχει διαθέσιμη	Η λίστα με τις ημερομηνίες φορτώνεται μόνο από τις διαθέσιμες της αίθουσας	Μέθοδος <code>fetchDates()</code> : καλεί <code>SeminarCalendar.getDates(room)</code>
3	Προσπάθεια αποθήκευσης με ελλιπή πεδία	Υποτίθεται ότι η GUI ελέγχει υποχρεωτικά πεδία πριν κλήση του <code>fetchSeminarRoom()</code>	Εκτός πεδίου ελέγχου του <code>seminarsMan.py</code> (πρέπει να γίνεται στην GUI)

4	Χρήστης αφαιρεί σεμινάριο χωρίς ενημέρωση	Γίνεται αυτόματα ειδοποίηση συμμετεχόντων	Μέθοδος <code>callControllerForDelete()</code> καλεί <code>NotificationService.notifyParticipants()</code>
5	Χρήστης δημιουργεί νέο σεμινάριο χωρίς αίθουσες	Ενημερώνεται με <code>Message5Screen</code> και δεν συνεχίζεται η διαδικασία	Μέθοδος <code>fetchSeminarRoom()</code> ελέγχει λίστα αιθουσών και αποτρέπει
6	Χρήστης αποθηκεύει σεμινάριο αλλά το slot μένει διαθέσιμο	Η διαθεσιμότητα ενημερώνεται αυτόματα	<code>saveSeminar()</code> καλεί <code>SeminarRoom.updateRoomAvailability()</code> και αφαιρεί slot

Test case 6: Διαχείριση Αιτημάτων για Παντεβού (discharge & discharge_gui αρχεία)

Η λειτουργία εξιτηρίου υλοποιείται με χρήση MVC και Tkinter GUI. Ο `ViewParentScreen` εμφανίζει λίστα γονέων και ελέγχει για εκκρεμείς πληρωμές μέσω `checkForPendingPayments()`. Αν υπάρχουν, εμφανίζεται `ViewPaymentsScreen`. Αν όχι, ενεργοποιείται το `DischargeScreen`, το οποίο επιτρέπει την έκδοση εξιτηρίου. Η δημιουργία εγγραφής εξιτηρίου γίνεται μέσω `Discharge.createDischarge()`, ενώ εμφανίζεται επιβεβαίωση στον χρήστη με `messagebox.showinfo()`. Η ροή αποτρέπει εξιτήριο σε περιπτώσεις οικονομικών εκκρεμοτήτων.

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη απο το να το κάνει	Μηχανισμός Αποτροπής στον κώδικα
1	Ο χρήστης προσπαθεί να εκδώσει εξιτήριο σε γονέα με οφειλές	Η διαδικασία εξιτηρίου μπλοκάρεται και εμφανίζεται λίστα πληρωμών	Στη <code>chooseParent()</code> , αν υπάρχουν <code>pending</code> , καλείται <code>foundPending()</code> και εμφανίζεται <code>ViewPaymentsScreen</code>
2	Ο χρήστης δεν επιλέγει γονέα	Δεν μπορεί να γίνει γιατί το dropdown έχει πάντα default τιμή	<code>self.parent_var.set(self.parents[0])</code> στην <code>displayParents()</code>
3	Ο χρήστης προσπαθεί να εκδώσει ξανά εξιτήριο	Δεν υπάρχει έλεγχος – μπορεί να βελτιωθεί	Στο <code>Discharge.createDischarge()</code> δεν γίνεται duplicate check (πιθανό future test)

Test case 7: Διαχείριση Αιτημάτων για Παντεβού (αρχεία doctorAppointment_gui, doctorAppointment)

Ο πίνακας καταγράφει βασικά test cases που σχετίζονται με την αποτροπή λανθασμένων ενεργειών χρήστη στο Doctor Appointment GUI. Η εφαρμογή χρησιμοποιεί ελέγχους εγκυρότητας (`if not selected`, `if appointment`) και καταστροφή προηγούμενων frames (`destroy()`) για να αποτρέψει μη επιτρεπτές ή εσφαλμένες ενέργειες κατά την πλοήγηση και διαχείριση ραντεβού.

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη απο το να το κάνει	Μηχανισμός Αποτροπής στον κώδικα
1	Ο χρήστης πατά "View Details" χωρίς να έχει επιλέξει ραντεβού	Δεν γίνεται μετάβαση – δεν εμφανίζεται σφάλμα	<code>if not selected: return</code> στο <code>chooseAppointmentReq()</code>
2	Ο Χρήστης επιλέγει ραντεβού που δεν υπάρχει (μπορεί να διαγράφηκε πιο πριν)	Η οθόνη δεν θα αλλάξει, δηλαδή το αίτημα θα αγνοηθεί	<code>if appointment:</code> στο <code>fetchAppointmentDetails()</code>
3	Ο χρήστης παταέι Accept ή Reject και δεν ξέρει αν το αποδέχτηκε η εφαρμογή (δεν έχει ορατό αποτέλεσμα η ενεργειά του)	Εμφανίζεται στην οθόνη ένα μήνυμα επιβεβαίωσης ενέργειας	<code>ConfirmScreen</code> εμφανίζει μήνυμα <code>Appointment {status} successfully!</code>
4	Ο χρήστης ανοίγει πολλές οθόνες μαζί ή τις εναλλάσσει πολύ γρήγορα	Πριν φορτώσει η νέα οθόνη, φροντίζουμε η παλιά να έχει καταστραφεί	<code>self.current_screen.destroy()</code> πριν το <code>pack()</code> κάθε νέας οθόνης
5	Ο χρήστης προσπαθεί να συνεχίσει χωρίς αλληλεπίδραση με την λίστα	Αναγκάζεται να επιλέξει κάτι – το κενό απαγορεύεται	<code>Treeview.focus()</code> Έλεγχος κενού selection

Test Case 8: Ενημέρωση Ιατρικού Φακέλου (αρχεία `medicalRecord_gui`, `medRecord`)

Η εφαρμογή υποστηρίζει προβολή και διαχείριση ιατρικού φακέλου με Tkinter και χρήση MVC αρχιτεκτονικής. Ο `MedicalRecordController` χειρίζεται την ανάκτηση και ενημέρωση

του φακέλου μέσω του μοντέλου `MedicalRecord`. Η αναζήτηση γονέα γίνεται με βάση ονοματεπώνυμο και εάν βρεθεί, εμφανίζεται η οθόνη `MedicalRecordScreen` με δυνατότητα προβολής σημειώσεων, προσθήκης νέας σημείωσης, συνταγογράφησης και ανάθεσης εξετάσεων. Κενές σημειώσεις αποτρέπονται με έλεγχο (`if not new_note`) και επιβεβαιώνονται μέσω `NoteConfirmationScreen`. Η αποθήκευση γίνεται με `record.updateMedicalRecord(note, parent, doctor)`, εξασφαλίζοντας σύνδεση γιατρού και γονέα

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη απο το να το κάνει	Μηχανισμός Αποτροπής στον κώδικα
1 *	Ο χρήστης εισάγει όνομα/επώνυμο που δεν αντιστοιχεί σε υπάρχοντα γονέα	Εμφανίζεται μήνυμα σφάλματος και δεν προχωράει	<code>fetchMedicalRecord()</code> επιστρέφει <code>False</code> , οδηγεί σε <code>messagebox.showerror()</code>
2	Ο χρήστης προσπαθεί να αποθηκεύσει σημείωση χωρίς κείμενο	Εμφανίζεται προειδοποιητικό μήνυμα και αποτρέπεται η αποθήκευση	Στη <code>addNewNote()</code> , αν το κείμενο είναι κενό, εμφανίζεται <code>messagebox.showwarning()</code>
3	Ο χρήστης αρνείται την επιβεβαίωση αποθήκευσης	Η σημείωση δεν αποθηκεύεται	Η <code>NoteConfirmationScreen</code> καλεί <code>saveNote()</code> μόνο αν ο χρήστης πατήσει "Ναι"
4	Ο χρήστης γράφει σημείωση και επιβεβαιώνει την αποθήκευση	Η σημείωση αποθηκεύεται στον φάκελο και ενημερώνεται το αντικείμενο	<code>saveNote()</code> καλεί <code>record.updateMedicalRecord()</code>
5	Ο χρήστης προσπαθεί να συνταγογραφήσει χωρίς αναζήτηση γονέα	Δεν είναι εφικτό, γιατί το κουμπί εμφανίζεται μόνο μετά την αναζήτηση	Το <code>PrescriptionController</code> δέχεται <code>parent</code> μόνο αφού εμφανιστεί <code>MedicalRecordScreen</code>
6	Ο χρήστης πατάει "Ανάθεση Εξετάσεων" χωρίς ενεργό parent	Δεν είναι δυνατόν – η λειτουργία ενεργοποιείται μόνο αφού έχει φορτωθεί ο φάκελος	Η <code>ExaminationController</code> λαμβάνει τον γονέα από <code>self.parent</code> στο <code>MedicalRecordScreen</code>

*

Για να είναι επιτυχημένη η αναζήτηση θα πρέπει τα πεδία να περιέχουν κάποιες από τις εξής τιμές:

Όνομα: Μαρία Επώνυμο: Παπαδοπούλου

Όνομα: Ελένη Επώνυμο: Πατπά

Αλλιώς δεν θα βρεθεί ποτέ ιατρικός φάκελος ασθενή.

Test case 9: Συνταγογράφηση (αρχεία prescrMed_gui, presMed)

Η εφαρμογή υλοποιεί διαχείριση συνταγογράφησης με MVC αρχιτεκτονική χρησιμοποιώντας Tkinter. Ο `PrescriptionController` φέρνει τη λίστα φαρμάκων (`Medicine.getMedicine()`), ελέγχει την εγκυρότητα δοσολογίας και συντονίζει την αποδοχή ή απόρριψη συνταγής μέσω GUI οθονών (`MedicineScreen`, `CreatePrescriptionScreen`). Η αποθήκευση της συνταγής γίνεται με δημιουργία αντικειμένου `Prescription`, ενημερώνεται το ιατρικό αρχείο (`MedicalRecord.updateMedicalRecordM`) και αποστέλλεται ειδοποίηση στον γονέα μέσω `NotificationService`. Λανθασμένες ενέργειες (π.χ. κενή δοσολογία) αποτρέπονται με εμφανίσεις προειδοποιητικών μηνυμάτων και αποκλεισμό αποθήκευσης.

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη απο το να το κάνει	Μηχανισμός Αποτροπής στον κώδικα
1	Ο χρήστης προσπαθεί να αποθηκεύσει συνταγή χωρίς να εισάγει δοσολογία	Δεν προχωράμε στην αποθήκευση και εμφανίζουμε μήνυμα σφάλματος	Μέθοδος <code>checkDosage()</code> ελέγχει αν το πεδίο δοσολογίας είναι κενό και καλεί <code>Message11Screen</code> για ενημέρωση
2	Ο χρήστης επιλέγει να ακυρώσει την αποθήκευση της συνταγής	Επιστροφή στην οθόνη επιλογής φαρμάκου χωρίς αποθήκευση	Στο <code>CreatePrescriptionScreen</code> , το κουμπί "Ακύρωση" καλεί <code>acceptOrRejectPrescription()</code> με <code>save=False</code> και εμφανίζει <code>RejectPrescriptionScreen</code>
3	Ο χρήστης προσπαθεί να επιλέξει φάρμακο που δεν υπάρχει στη λίστα	Δεν εμφανίζεται στο dropdown, άρα δεν είναι επιλέξιμο	Η λίστα φαρμάκων φορτώνεται από <code>Medicine.getMedicine()</code> και το GUI γεμίζει μόνο με αυτά
4	Ο χρήστης εισάγει σωστή δοσολογία και επιλέγει αποθήκευση	Ενημερώνεται το ιατρικό αρχείο και στέλνεται ειδοποίηση στον γονέα	<code>acceptOrRejectPrescription()</code> δημιουργεί <code>Prescription</code> , ενημερώνει <code>MedicalRecord</code> και καλεί <code>NotificationService</code>

Test Case 10 : Διεξαγωγή Εξετάσεων (αρχεία examMed.py, examMed_gui.py)

Η λειτουργία ανάθεσης εξετάσεων υλοποιείται με MVC και Tkinter GUI. Ο `ExaminationController` χειρίζεται την επιλογή εξετάσεων, υπολογισμό κόστους και αποθήκευση μέσω της `Examination`. Αν ο χρήστης επιλέξει ήδη καταχωρημένες εξετάσεις, η μέθοδος `checkScheduledExams()` ενεργοποιεί την

`ScheduledExaminationScreen`, αποτρέποντας διπλή ανάθεση. Επιπλέον, η αποδοχή εξετάσεων απαιτεί τουλάχιστον μία επιλογή και αποτρέπεται πλήρως αν το `selected_exams` είναι κενό. Οι ακυρώσεις αποθηκεύσεων οδηγούν σε `RejectExaminationScreen`, χωρίς καμία καταχώρηση στο σύστημα.

Test Case	Περιγραφή	Πως αποτρέπουμε τον χρήστη απο το να το κάνει	Μηχανισμός Αποτροπής στον κώδικα
1	Ο χρήστης προσπαθεί να συνεχίσει χωρίς να έχει επιλέξει κάποια εξέταση	Εμφανίζεται <code>messagebox.showerror()</code> και μπλοκάρεται η συνέχεια	Στο <code>chooseExams()</code> : <code>if not selected_exams</code> : εμφανίζει σφάλμα και επιστρέφει
2	Ο χρήστης επιλέγει εξετάσεις που έχουν ήδη ανατεθεί	Εμφανίζεται οθόνη ενημέρωσης <code>ScheduledExaminationScreen</code> και δεν επιτρέπεται αποθήκευση	Η <code>sendChosenExams()</code> καλεί <code>checkScheduledExams()</code> και εμφανίζει ξεχωριστή οθόνη
3	Ο χρήστης πατά "Ακύρωση" μετά την επιλογή εξετάσεων	Η εγγραφή εξετάσεων δεν αποθηκεύεται, επιστρέφει χωρίς ενέργεια	<code>CreateExaminationScreen.rejectExams()</code> καλεί <code>acceptOrReject(..., save=False)</code> και εμφανίζει <code>RejectExaminationScreen</code>
4	Ο χρήστης προσπαθεί να επιλέξει ξανά ήδη καταχωρημένη εξέταση	Δεν είναι δυνατόν, γιατί η επιλογή αποτρέπεται από τον έλεγχο state	Οι ήδη προγραμματισμένες εξετάσεις φιλτράρονται από το <code>checkScheduledExams()</code>